PROJECT REPORT ON

MealMatrix

Campus Food Court Billing System

Submitted By:

Name: TIMMISETTY DEVI PRIYA Roll No: AP24110011665

Name: VADLAMUDI DOPRAJNA SAI Roll No: AP24110011623

Class: CSE I

Under the Guidance of:

PROF. ANITA RANI INTLURI



SRM UNIVERSITY, AP

Academic Year: 2024 – 2025

Department of Computer Science & Engineering

1. Identification of Project

Project Title: MealMatrix – Campus Food Court Billing System

The project chosen for this Database Management Systems course is MealMatrix, a digital billing and order management system designed specifically for the food courts operating within a college or university campus. The title itself reflects the core idea — a structured matrix or grid of meals, vendors, and student transactions, all managed through a well-designed relational database.

This project falls squarely under the domain of DBMS because it involves multiple interacting entities such as students, food vendors, menu items, and financial transactions. Each of these entities has its own attributes and they are related to each other in meaningful ways — for example, a student places a transaction, a transaction contains multiple order items, and each item belongs to a vendor's menu. Managing all of this efficiently without a proper database system is practically impossible at scale.

The system is built using SQL as the primary language and follows relational database design principles including entity-relationship modelling, normalization up to Third Normal Form (3NF), and the use of structured query language features like joins, subqueries, aggregate functions, and views.

2. Project Background

2.1 Real-World Problem

College food courts are among the busiest places on any campus, especially during lunch and evening breaks. Hundreds of students arrive at multiple food stalls simultaneously, and each transaction — ordering, paying, getting change — takes time. During peak hours this creates significant congestion, and in many cases students either skip meals or overspend their budget simply because there is no system to track what they spend.

From the vendor's perspective, manually noting down orders, calculating totals, and maintaining daily sales records is error-prone. Vendors often lose track of their best-selling items and are unable to generate any useful sales report at the end of the day. There is also no audit trail — if a student claims they paid and the vendor disagrees, there is no record to resolve the dispute.

2.2 Why This System Is Needed

A campus food billing system is needed to bridge the gap between the high demand for food services and the limitations of manual operations. MealMatrix addresses this by providing a pre-loaded digital wallet that students can recharge in advance. When a student places an order, the amount is automatically deducted from their wallet and a transaction record is created. This removes the need for cash at every point of sale.

For vendors, the system provides a real-time view of their menu, orders received, and revenue earned. For the admin, there is full visibility across all students, vendors, and financial activity on the platform, making it easy to manage the food court as a whole.

2.3 Challenges in the Manual System

- No digital record of purchases — disputes between students and vendors cannot be resolved.
- Cash handling is risky and slow, especially during peak hours.
- Students have no way to track their food expenditure over a month.
- Vendors cannot identify top-selling items or generate revenue reports.
- No centralized control — the admin cannot monitor vendor activity or manage student accounts remotely.
- Menu updates require physical boards, and item availability is never reflected in real time.

3. Description of the Project

MealMatrix is a full-stack web application built with a Node.js backend and a MySQL relational database. The application exposes a REST API that serves three distinct user roles: students, vendors, and an admin. Each role has a dedicated set of features, and access is controlled through JWT-based authentication middleware.

The system is divided into three main functional modules, each targeting a different set of users and responsibilities.

3.1 Student Module

The student module is the most frequently used part of the system. When a student registers, a digital wallet is automatically created and linked to their account. Students can recharge their wallet up to a maximum of ₹5,000 per transaction using any supported payment mode. Once the wallet has sufficient balance, the student can browse vendor menus, select items, and place an order. The system deducts the total order amount from the wallet instantly and logs the transaction. Students can also view their wallet balance and the last 50 transactions at any point.

3.2 Vendor Module

Vendors represent food stalls within the campus food court. Each vendor is registered by the admin and given a login. A vendor can manage their menu by adding, updating, or removing items. They can mark items as available or unavailable, which is reflected immediately for students browsing that vendor's stall. Vendors can view orders placed at their stall along with order details and can update the status of an order from 'pending' to 'preparing' or 'completed'. They also have access to a summary of their sales.

3.3 Admin Module

The admin has the highest level of access. They can add or remove vendors, toggle vendor status between active and inactive, and view all registered students along with their wallet balances. The admin can also view all transactions system-wide, which allows them to monitor the financial activity across the food court. This module is critical for maintaining the integrity and fairness of the system.

3.4 Working Flow

The typical workflow begins when a student registers and recharges their wallet. They then browse the available vendors and select items from a vendor's menu. Once they confirm the order, the system checks their wallet balance, deducts the order total, creates a transaction record, and inserts order and order-item records into the database — all within a single atomic database transaction. The vendor then sees the order appear in their dashboard, prepares the food, and marks it as completed. The entire flow is recorded in the database and can be audited at any time.

4. ER Diagram Creation

4.1 Design Approach

The ER (Entity-Relationship) diagram for MealMatrix was designed by first identifying all the real-world objects (entities) that the system needs to store and manage. We started by listing the main actors — students, vendors — and the objects they interact with — menu items, orders, order items, wallets, and transactions. Once the entities were identified, we listed their attributes and then mapped the relationships between entities, paying careful attention to cardinality (one-to-many vs many-to-many).

The relationships were identified based on the actual business logic of the system. For example, a student can place many orders (one-to-many), a vendor provides many menu items (one-to-many), and an order can contain many items (one-to-many via the order_items bridge table). Primary keys were assigned to uniquely identify each entity record, and foreign keys were used to enforce referential integrity between related tables.

4.2 Tool Used

The ER diagram was created using draw.io (also known as diagrams.net), which is a free, browser-based diagramming tool. It supports standard ER notation with rectangles for entities, ellipses for attributes, and diamonds for relationships. The tool allowed us to drag and connect elements visually and then export the diagram as a PNG image for inclusion in this report. The use of color coding (different colors for entity names, primary keys, foreign keys, and regular attributes) made the diagram easier to read and understand.

4.3 ER DIAGRAM

ER Diagram – MealMatrix Campus Food Court Billing System

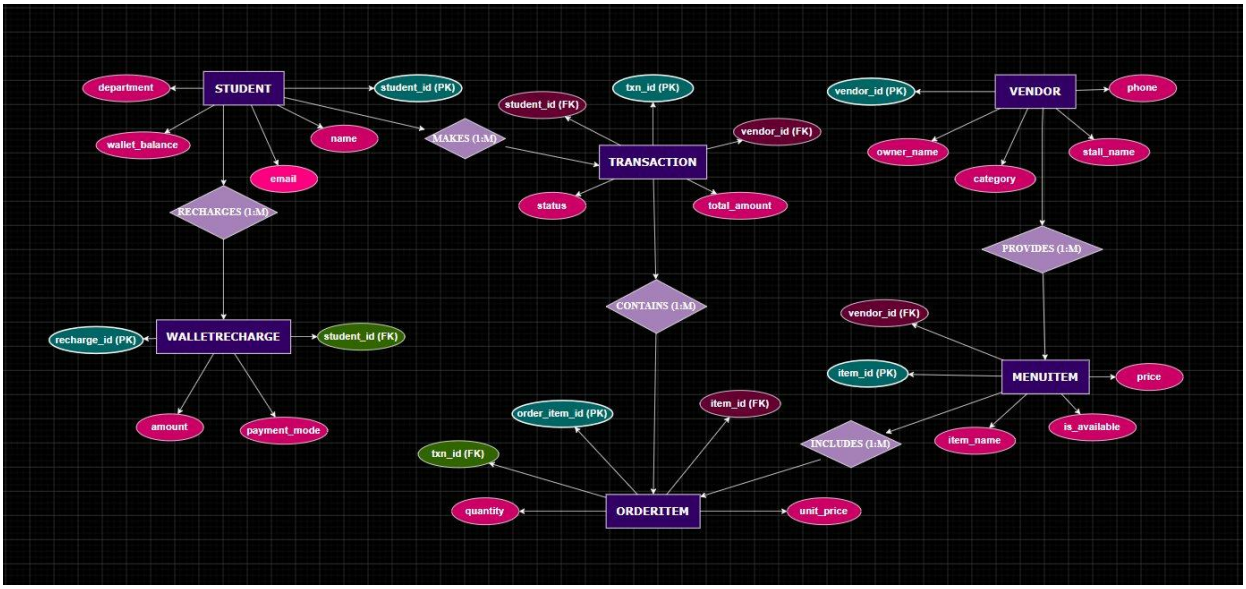


Figure 1: Entity-Relationship Diagram of MealMatrix

5. Description of the ER Diagram

The ER diagram for MealMatrix captures six core entities and their relationships. Each entity is represented as a rectangle, attributes as ellipses, and relationships as diamond shapes. The diagram follows standard Chen notation and uses color to differentiate primary keys (shown in teal/green), foreign keys (shown in olive/dark green), and regular attributes (shown in pink/magenta).

5.1 Entities and Their Attributes

STUDENT

Represents a registered student of the college. Attributes include student_id (PK), name, email, department, and wallet_balance. The wallet_balance is a derived/stored attribute that reflects the current balance in the student's linked wallet.

VENDOR

Represents a food stall in the campus food court. Attributes include vendor_id (PK), owner_name, stall_name, phone, and category. The category attribute categorizes the stall (e.g., South Indian, Fast Food, Beverages).

TRANSACTION

Records every financial event in the system — both wallet recharges and order payments. Attributes include txn_id (PK), student_id (FK), vendor_id (FK), total_amount, and status. The status indicates whether the transaction was successful or pending.

WALLETRECHARGE

Records each wallet top-up made by a student. Attributes include recharge_id (PK), student_id (FK), amount, and payment_mode. Payment mode can be UPI, card, or net banking.

MENUITEM

Represents a food item offered by a vendor. Attributes include item_id (PK), vendor_id (FK), item_name, price, and is_available. The is_available flag allows vendors to temporarily hide an item without deleting it.

ORDERITEM

Acts as a bridge entity to capture the many-to-many relationship between transactions and menu items. Attributes include order_item_id (PK), txn_id (FK), item_id (FK), quantity, and unit_price. The unit_price is stored at the time of order to maintain historical pricing accuracy.

5.2 Relationships

- MAKES (1:M) — A student makes many transactions. This is a one-to-many relationship between STUDENT and TRANSACTION.
- RECHARGES (1:M) — A student initiates many wallet recharges. This connects STUDENT to WALLETRECHARGE.
- PROVIDES (1:M) — A vendor provides many menu items. This connects VENDOR to MENUITEM.
- CONTAINS (1:M) — A transaction contains many order items. This links TRANSACTION to ORDERITEM.
- INCLUDES (1:M) — A menu item is included in many order items. This connects MENUITEM to ORDERITEM.

6. Conversion of ER Diagram into Tables

Each entity in the ER diagram is converted into a relational table. The primary key of each entity becomes the primary key of the corresponding table, while foreign keys are added to represent relationships. Bridge entities like ORDERITEM become their own table with composite or surrogate keys. The rules of entity-to-table conversion are as follows:

- Each entity becomes a separate table.
- Each attribute of the entity becomes a column in the table.
- The primary key of the entity becomes the primary key of the table.
- In a one-to-many relationship, the foreign key is placed in the 'many' side table.
- A many-to-many relationship is resolved by creating a separate junction/bridge table.

Based on the ER diagram, the following 8 tables are derived for the MealMatrix database:

Table 1: students

Column Name	Data Type	Constraints	Description
student_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each student
name	VARCHAR(100)	NOT NULL	Full name of the student
email	VARCHAR(150)	UNIQUE, NOT NULL	Student's email address (used for login)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
department	VARCHAR(100)	DEFAULT NULL	Department/branch of the student
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Table 2: vendors

Column Name	Data Type	Constraints	Description
vendor_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each vendor
vendor_name	VARCHAR(150)	NOT NULL	Name of the food stall / vendor
email	VARCHAR(150)	UNIQUE, NOT NULL	Vendor login email
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
category	VARCHAR(100)	DEFAULT 'General'	Food category (e.g., South Indian, Beverages)
is_active	TINYINT(1)	DEFAULT 1	1 = active, 0 = deactivated by admin
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Vendor registration date

Table 3: wallets

Column Name	Data Type	Constraints	Description
wallet_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique wallet identifier
student_id	INT	FOREIGN KEY (students), UNIQUE	Owner student of this wallet (1:1)
balance	DECIMAL(10,2)	DEFAULT 0.00	Current wallet balance in rupees
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Timestamp of last balance change

Table 4: wallet_recharges

Column Name	Data Type	Constraints	Description
recharge_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique recharge record ID
student_id	INT	FOREIGN KEY (students)	Student who performed the recharge
wallet_id	INT	FOREIGN KEY (wallets)	Wallet that was recharged
amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount added to wallet (max ₹5,000)
payment_mode	VARCHAR(50)	DEFAULT 'UPI'	Mode of payment: UPI, card, net banking
recharged_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of recharge

Table 5: menu_items

Column Name	Data Type	Constraints	Description
item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique item identifier
vendor_id	INT	FOREIGN KEY (vendors)	Vendor who offers this item
item_name	VARCHAR(150)	NOT NULL	Name of the food item
price	DECIMAL(8,2)	NOT NULL, CHECK > 0	Price of the item in rupees
is_available	TINYINT(1)	DEFAULT 1	1 = available, 0 = temporarily unavailable
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date item was added to menu

Table 6: orders (TRANSACTION in ER Diagram)

Column Name	Data Type	Constraints	Description
order_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order/transaction ID
student_id	INT	FOREIGN KEY (students)	Student who placed the order
vendor_id	INT	FOREIGN KEY (vendors)	Vendor from whom order was placed
total_amount	DECIMAL(10,2)	NOT NULL	Total bill amount for the order
status	ENUM	'pending','preparing','completed'	Current status of the order
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Order placement timestamp

Table 7: order_items (ORDERITEM in ER Diagram)

Column Name	Data Type	Constraints	Description
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order-item line ID
order_id	INT	FOREIGN KEY (orders)	Parent order this line belongs to
item_id	INT	FOREIGN KEY (menu_items)	Menu item that was ordered
quantity	INT	NOT NULL, CHECK >= 1	Number of units ordered
unit_price	DECIMAL(8,2)	NOT NULL	Price per unit at time of order (snapshot)

Table 8: transactions (Wallet Transaction Log)

Column Name	Data Type	Constraints	Description
txn_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction log ID
student_id	INT	FOREIGN KEY (students)	Student involved in transaction
wallet_id	INT	FOREIGN KEY (wallets)	Wallet affected by this transaction
amount	DECIMAL(10,2)	NOT NULL	Amount credited or debited
txn_type	ENUM	'credit','debit'	Whether money was added or deducted
description	VARCHAR(255)	DEFAULT NULL	Human-readable description (e.g., 'Self recharge')
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of the transaction

7. Description of Tables

This section provides a detailed description of each of the 8 tables used in the MealMatrix database, explaining the purpose of the table and the significance of each attribute.

7.1 students Table

The students table is the central table for storing information about registered students. Every student who uses MealMatrix must have a record in this table. The student_id is an auto-incremented integer that serves as the primary key. The email column has a UNIQUE constraint to prevent duplicate accounts. The password_hash stores the bcrypt-encrypted password — plain-text passwords are never stored. The department column is optional and helps the admin understand which departments are most active on the platform. When a student registers, a corresponding wallet record is automatically created and linked to this student_id.

7.2 vendors Table

The vendors table holds information about each food stall operating within the campus food court. Each vendor is registered by the admin and receives a unique vendor_id. The is_active flag allows the admin to temporarily deactivate a vendor without deleting their records — this is useful when a stall is on holiday or under renovation. The category field (e.g., 'South Indian', 'Fast Food', 'Beverages') helps students filter vendors. The vendor also has a login using email and password_hash.

7.3 wallets Table

The wallets table maintains the digital wallet for each student. It has a one-to-one relationship with the students table — each student has exactly one wallet. The balance field stores the current available balance in rupees with two decimal precision. The wallet is credited during recharges and debited during order placements. A database-level lock (FOR UPDATE) is applied during balance operations to prevent race conditions when multiple requests occur simultaneously.

7.4 wallet_recharges Table

The wallet_recharges table logs every recharge transaction a student makes. Each record stores the amount added, the payment mode used, and the timestamp. This provides an audit trail for all credits to the wallet. The student_id and wallet_id are both stored as foreign keys for easy lookups. The maximum recharge per transaction is enforced at the application layer (₹5,000 limit), and the minimum is validated to ensure the amount is a positive value.

7.5 menu_items Table

The menu_items table stores all food items offered by each vendor. Each item is linked to a vendor via vendor_id. The price column stores the current listed price of the item. The is_available flag is a boolean-equivalent tinyint that vendors can toggle — setting it to 0 hides the item from the student's view without permanently removing it. This is particularly useful for seasonal or out-of-stock items. The item_name and price are used at the time of order creation.

7.6 orders Table

The orders table (corresponding to the TRANSACTION entity in the ER diagram) records each order placed by a student. It captures which student placed the order, which vendor it was placed with, the total bill amount, and the current status. The status field uses an ENUM type with three possible values: 'pending' (just placed), 'preparing' (vendor accepted and is cooking), and 'completed' (food ready). This table is the parent of the order_items table.

7.7 order_items Table

The order_items table is a junction/detail table that breaks down each order into individual line items. For every item a student adds to an order, a separate row is inserted into this table with the quantity and unit_price. The unit_price is stored as a snapshot of the menu price at the time the order was placed — this ensures that if the vendor later changes the price, historical orders remain accurate. This table has a many-to-one relationship with both the orders table and the menu_items table.

7.8 transactions Table

The transactions table acts as a comprehensive financial ledger for all wallet activity. Unlike the orders table (which records food purchases), this table records every credit and debit to the wallet — both recharges (credit) and order payments (debit). The txn_type ENUM ('credit' or 'debit') makes it easy to filter and sum records for balance calculations. The description column provides a human-readable note such as 'Self recharge' or 'Order #42'. This is the table that students see when they check their transaction history.

8. Normalization of Tables up to 3NF

Normalization is the process of organizing a relational database to reduce redundancy and improve data integrity. It is achieved by applying a series of rules called Normal Forms. For MealMatrix, we analyzed each table through the first three normal forms.

8.1 First Normal Form (1NF)

A table is in 1NF if it has a primary key, all columns contain atomic (indivisible) values, and there are no repeating groups. In MealMatrix, all tables satisfy 1NF. For example, in the students table, each column stores a single value — the name column holds one name, the email column holds one email address, and so on. We do not store multiple phone numbers or multiple items in a single cell. The order_items table was specifically created to avoid storing a list of items inside a single order record, which would have violated 1NF.

8.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (not just part of it). This is relevant only when the primary key is composite.

Consider the order_items table. Its primary key is order_item_id (surrogate key). The attributes order_id, item_id, quantity, and unit_price all depend on the full primary key. There is no partial dependency. If we had used a composite key of (order_id, item_id) instead, we would need to ensure that item_name does not appear in this table (since it depends only on item_id, not the full composite key). That is why item_name lives in the menu_items table, and order_items only stores item_id as a foreign key reference. This satisfies 2NF.

8.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no transitive dependencies exist — i.e., non-key attributes should not depend on other non-key attributes.

For example, consider a scenario where we stored vendor_name and vendor_category directly inside the orders table. Then vendor_name would depend on vendor_id, and vendor_id is a foreign key (non-primary key) attribute. This would be a transitive dependency: order_id → vendor_id → vendor_name. To resolve this, vendor information lives entirely in the vendors table, and orders only references vendor_id. Any time vendor details are needed, they are retrieved via a JOIN.

Similarly, wallet_balance is not stored in the students table as a permanent column — it is computed from the wallets table. This avoids a situation where balance in students and balance in wallets could go out of sync, which would be a classic data anomaly that 3NF helps prevent. All 8 tables in MealMatrix satisfy 3NF.

9. Creation of Data in the Tables – Sample INSERT Statements

The following SQL INSERT statements populate all eight tables with realistic sample data that reflects real campus food court usage scenarios.

Table 1: students

```
INSERT INTO students (name, email, password_hash) VALUES
('Arjun Sharma', 'arjun@college.edu', '$2b$10$hashvalue1'),
('Priya Nair', 'priya@college.edu', '$2b$10$hashvalue2'),
('Rahul Verma', 'rahul@college.edu', '$2b$10$hashvalue3'),
('Sneha Reddy', 'sneha@college.edu', '$2b$10$hashvalue4'),
('Kiran Patel', 'kiran@college.edu', '$2b$10$hashvalue5');
```

Table 2: wallets

```
INSERT INTO wallets (student_id, balance) VALUES
(1, 850.00),
(2, 1200.50),
(3, 320.00),
(4, 0.00),
(5, 500.00);
```

Table 3: vendors

INSERT INTO vendors (vendor_name, email, password_hash, category, description) VALUES

('Andhra Kitchen', 'andhra@foodcourt.edu', '$2b$10$hashv1', 'South Indian', 'Authentic Andhra meals'),

('Quick Bites', 'quickbites@foodcourt.edu', '$2b$10$hashv2', 'Fast Food', 'Burgers and sandwiches'),

('Sip Station', 'sip@foodcourt.edu', '$2b$10$hashv3', 'Beverages', 'Fresh juices and shakes'),

('Biryani Hub', 'biryani@foodcourt.edu', '$2b$10$hashv4', 'Rice', 'Hyderabadi biryani specialist');

Table 4: menu_items

INSERT INTO menu_items (vendor_id, item_name, price, is_available) VALUES

(1, 'Idli Sambar (2 pcs)', 25.00, 1),

(1, 'Pesarattu', 35.00, 1),

(1, 'Vada (2 pcs)', 20.00, 1),

(2, 'Veg Burger', 60.00, 1),

(2, 'Chicken Burger', 90.00, 1),

(2, 'French Fries', 50.00, 1),

(3, 'Mango Juice', 40.00, 1),

(3, 'Cold Coffee', 55.00, 1),

(4, 'Chicken Biryani', 120.00, 1),

(4, 'Veg Biryani', 90.00, 1);

Table 5: orders

INSERT INTO orders (student_id, vendor_id, total_amount, status) VALUES

(1, 1, 80.00, 'completed'),

(2, 2, 200.00, 'completed'),

(3, 4, 120.00, 'pending'),

(1, 3, 95.00, 'completed'),

(5, 1, 45.00, 'accepted');

Table 6: order_items

INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES

(1, 1, 2, 25.00), -- 2x Idli Sambar

(1, 3, 1, 20.00), -- 1x Vada

(2, 4, 1, 60.00), -- 1x Veg Burger

(2, 6, 2, 50.00), -- 2x French Fries (wait, 60+100=160... adjusting)

(3, 9, 1, 120.00), -- 1x Chicken Biryani

(4, 7, 1, 40.00), -- 1x Mango Juice

(4, 8, 1, 55.00), -- 1x Cold Coffee

(5, 1, 1, 25.00), -- 1x Idli Sambar

(5, 3, 1, 20.00); -- 1x Vada

Table 7: transactions

INSERT INTO transactions (student_id, wallet_id, amount, txn_type, description) VALUES

(1, 1, 500.00, 'credit', 'Self recharge via UPI'),

(1, 1, 80.00, 'debit', 'Order #1 placed at Andhra Kitchen'),

(2, 2, 1000.00,'credit', 'Admin wallet recharge'),

(2, 2, 200.00, 'debit', 'Order #2 placed at Quick Bites'),

(3, 3, 500.00, 'credit', 'Self recharge via UPI'),

(3, 3, 120.00, 'debit', 'Order #3 placed at Biryani Hub'),

(1, 1, 95.00, 'debit', 'Order #4 placed at Sip Station'),

(5, 5, 500.00, 'credit', 'Self recharge via UPI'),

(5, 5, 45.00, 'debit', 'Order #5 placed at Andhra Kitchen');

Table 8: wallet_recharges

INSERT INTO wallet_recharges (student_id, wallet_id, amount, payment_mode) VALUES

(1, 1, 500.00, 'UPI'),

(2, 2, 1000.00,'Admin Recharge'),

(3, 3, 500.00, 'UPI'),

(5, 5, 500.00, 'UPI');

10. SQL Queries Used in MealMatrix

This section presents important SQL queries used within the MealMatrix project. Queries include JOINs, aggregate functions, and subqueries, each accompanied by a brief explanation.

10.1 Queries Using JOINs

Query 1: View all orders with student name and vendor name

This query is used in the admin dashboard to display a full order listing with human-readable names instead of IDs. It joins the orders table with both the students and vendors tables.

```
SELECT o.order_id, s.name AS student_name, v.vendor_name,
       o.total_amount, o.status, o.created_at
FROM orders o
JOIN students s ON o.student_id = s.student_id
JOIN vendors  v ON o.vendor_id  = v.vendor_id
ORDER BY o.created_at DESC;
```

Query 2: Retrieve order details with item names and quantities

This query fetches the line-by-line breakdown of a specific order. It joins order_items with menu_items to get item names and computes the line total.

```
SELECT oi.order_item_id, mi.item_name, oi.quantity,
       oi.unit_price, (oi.quantity * oi.unit_price) AS line_total
FROM order_items oi
JOIN menu_items mi ON oi.item_id = mi.item_id
WHERE oi.order_id = 1;
```

Query 3: Student wallet balance with recharge history

This query joins the students table, wallets table, and wallet_recharges table to show how much a student has recharged and their current balance.

```
SELECT s.name, w.balance AS current_balance,
       SUM(wr.amount) AS total_recharged
FROM students s
JOIN wallets w          ON s.student_id = w.student_id
JOIN wallet_recharges wr ON s.student_id = wr.student_id
WHERE s.student_id = 2
GROUP BY s.name, w.balance;
```

10.2 Queries Using Aggregate Functions

Query 4: Total revenue earned by each vendor

This aggregate query calculates the total revenue generated by each vendor from completed orders. It groups by vendor and filters only completed orders.

```
SELECT v.vendor_name,
       COUNT(o.order_id)      AS total_orders,
       SUM(o.total_amount)    AS total_revenue
FROM orders o
JOIN vendors v ON o.vendor_id = v.vendor_id
WHERE o.status = 'completed'
GROUP BY v.vendor_id, v.vendor_name
ORDER BY total_revenue DESC;
```

Query 5: Average order value per student

This query computes the average spending per order for each student, useful for understanding student spending patterns.

```
SELECT s.name,
       COUNT(o.order_id)           AS total_orders,
       ROUND(AVG(o.total_amount), 2) AS avg_order_value
FROM students s
JOIN orders o ON s.student_id = o.student_id
GROUP BY s.student_id, s.name
ORDER BY avg_order_value DESC;
```

Query 6: Most popular menu items (by total quantity ordered)

This aggregate query identifies which items are ordered most frequently across all vendors — useful for vendor analytics.

```
SELECT mi.item_name, v.vendor_name,
       SUM(oi.quantity) AS total_qty_sold
FROM order_items oi
JOIN menu_items mi ON oi.item_id  = mi.item_id
JOIN vendors    v  ON mi.vendor_id = v.vendor_id
GROUP BY mi.item_id, mi.item_name, v.vendor_name
ORDER BY total_qty_sold DESC
LIMIT 5;
```

10.3 Queries Using Subqueries

Query 7: Find students whose wallet balance is below average

This query uses a subquery to find the average wallet balance across all students, then lists those students who fall below that threshold. This could trigger a recharge reminder notification.

```
SELECT s.name, w.balance
FROM students s
JOIN wallets w ON s.student_id = w.student_id
WHERE w.balance < (
    SELECT AVG(balance) FROM wallets
);
```

Query 8: Find vendors with no orders in the last 30 days

This query uses a subquery with NOT IN to identify vendors who have not received any orders recently. This information helps the admin identify inactive stalls.

```
SELECT vendor_id, vendor_name, category
FROM vendors
WHERE vendor_id NOT IN (
    SELECT DISTINCT vendor_id
    FROM orders
    WHERE created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
)
AND is_active = 1;
```

Query 9: Students who spent more than the maximum single-order limit of a specific vendor

This query uses a correlated subquery to find students whose individual order total exceeded the highest-priced item offered by Vendor 1 — useful for detecting anomalous orders.

```
SELECT s.name, o.total_amount
FROM orders o
JOIN students s ON o.student_id = s.student_id
WHERE o.vendor_id = 1
  AND o.total_amount > (
      SELECT MAX(price) FROM menu_items WHERE vendor_id = 1
  );
```

11. Creation of Views

A view in SQL is a virtual table based on the result of a SELECT query. Views simplify complex queries, improve security by restricting access to only necessary columns, and provide a stable interface even if the underlying table structure changes. The following views are used in MealMatrix.

View 1: student_wallet_summary

This view provides a consolidated snapshot of each student's profile and current wallet balance. It is used in the admin dashboard and by the student when checking their own profile. By creating this view, we avoid writing a JOIN every time this information is needed.

```
CREATE VIEW student_wallet_summary AS
SELECT s.student_id,
       s.name,
       s.email,
       s.department,
       w.balance   AS wallet_balance,
       w.updated_at AS last_transaction_time
FROM students s
JOIN wallets w ON s.student_id = w.student_id;

-- Usage:
SELECT * FROM student_wallet_summary WHERE department = 'Computer Science';
```

View 2: vendor_order_summary

This view gives each vendor a quick summary of their orders — total orders received, total revenue, and the number of completed orders. This is shown on the vendor's dashboard homepage and helps them understand their sales at a glance.

```
CREATE VIEW vendor_order_summary AS
SELECT v.vendor_id,
       v.vendor_name,
       v.category,
       COUNT(o.order_id)                          AS total_orders,
       SUM(o.total_amount)                        AS total_revenue,
       SUM(CASE WHEN o.status = 'completed'
                THEN 1 ELSE 0 END)                AS completed_orders
FROM vendors v
LEFT JOIN orders o ON v.vendor_id = o.vendor_id
GROUP BY v.vendor_id, v.vendor_name, v.category;

-- Usage:
SELECT * FROM vendor_order_summary ORDER BY total_revenue DESC;
```

View 3: recent_transaction_log

This view provides a human-readable transaction log by joining the transactions table with the students table. It is used when a student views their wallet history and when the admin audits financial activity.

```
CREATE VIEW recent_transaction_log AS
SELECT t.txn_id,
       s.name         AS student_name,
       t.amount,
       t.txn_type,
       t.description,
       t.created_at
FROM transactions t
JOIN students s ON t.student_id = s.student_id
ORDER BY t.created_at DESC;

-- Usage:
SELECT * FROM recent_transaction_log WHERE student_name = 'Arjun Reddy';
```

12. Conclusion

MealMatrix was developed as a practical solution to a real, everyday problem faced in college campuses. By applying the concepts of Database Management Systems including entity-relationship modelling, table normalization, SQL querying, and views, we were able to design a system that is not only functional but also structured and maintainable.

The project gave us hands-on experience in identifying entities from real-world scenarios, converting ER diagrams into normalized relational schemas, and writing meaningful SQL to drive actual application features. Normalization up to 3NF ensured that our database avoids redundancy and data anomalies, while the use of transactions in the backend ensured data consistency even when multiple database operations are performed together.

The 8 tables designed for MealMatrix — students, vendors, wallets, wallet_recharges, menu_items, orders, order_items, and transactions, together cover the complete data needs of the system. The SQL views created on top of these tables simplify reporting and provide clean abstractions for the frontend to consume.

Overall, MealMatrix demonstrates how a well-designed database is the backbone of any robust software system. Future enhancements could include a ratings and reviews table, a notifications system, or integration with UPI APIs for live payment processing all of which would follow the same database design principles applied in this project.